

监控区域入侵检测实验指南

一、实验概述

1.1 实验背景

视频监控是物联网（IoT）安防领域的核心应用。传统监控仅录制画面、事后回看，缺乏**实时智能分析能力**。本实验结合**边缘计算 + 深度学习**，在摄像头端实时完成：画面采集、警戒区域划定、区域内目标检测、入侵告警输出，实现“**事前预警、事中告警、事后追溯**”的物联网安防闭环。

本实验基于 **PyTorch + OpenCV + 轻量 CNN**，实现**电脑摄像头实时监控 + 自定义警戒区域 + 区域内人体入侵检测 + 实时告警**，可直接部署于树莓派、边缘网关、智能摄像头等物联网设备。

1.2 实验目标

完成本实验后，学生应能够：

- 理解：**图像采集** → **警戒区域划定** → **区域裁剪** → **特征提取** → **神经网络分类** → **告警输出** 的完整流程。
- 掌握：PyTorch 视觉项目依赖安装、模型训练、摄像头实时推理、OpenCV 绘图与告警逻辑设计。
- 实现：**实时摄像头监控 + 自定义矩形警戒区域 + 区域内人体入侵检测**。
- 设计：入侵事件映射为物联网告警动作（弹窗提示、日志记录）。
- 完成：拓展实验、检测阈值调参、复杂背景鲁棒性分析、边缘部署可行性评估。

1.3 项目功能介绍

本项目实现**实时视频监控与区域入侵检测**：

类型	功能	识别结果
实时视频监控	调用本地摄像头，实时显示监控画面	正常画面 / 异常告警
自定义警戒区域	手动划定矩形禁区，仅检测区域内目标	区域内 / 区域外
人体入侵检测	CNN 识别区域内是否出现人体	Human Detected / No Intrusion
实时告警输出	入侵时弹窗提示、画面标红、日志记录	Warning / Safe

1.4 物联网应用场景

场景	应用描述
园区安防	厂区、小区、停车场禁区入侵告警
家庭监控	门窗、阳台警戒，陌生人闯入预警
工业安防	机房、设备区禁止人员进入检测
边缘 AI 网关	摄像头本地分析，仅上传告警事件，节省带宽
无人值守场景	仓库、变电站、基站远程智能监控

二、实验环境

2.1 推荐配置

- 操作系统：Windows 10/11
- Python：3.9-3.12
- 硬件：8GB 内存、内置 / 外接摄像头
- 主要依赖：PyTorch、TorchVision、OpenCV-Python、NumPy

2.2 依赖安装

```
pip install torch torchvision opencv-python numpy
```

2.3 目录结构

```
iot_surveillance/  
├─ main.py           # 主程序：摄像头监控+区域检测+告警  
├─ train.py          # CNN模型训练脚本  
├─ data/             # 监控场景图像数据集  
│   └─ image/ # 数据集  
├─ models/             
│   └─ best_model.pth # 训练好的PyTorch模型  
└─ logs/               
    └─ alarm_log.csv  # 入侵告警日志，在test的时候记录
```

核心文件说明：

文件	作用
main.py	主程序入口：摄像头读取、警戒区域绘制、模型推理、告警显示
train.py	训练轻量 CNN 模型，区分“有人 / 无人”
cnn_surveillance.pth	训练完成的 PyTorch 权重文件，用于实时推理
alarm_log.csv	记录告警时间、画面帧、检测结果，便于追溯

三、基础实验：实时监控与区域入侵检测

3.1 系统处理流程

```
摄像头实时采集 → OpenCV读取视频帧 → 绘制自定义警戒矩形框 →裁剪警戒区域ROI → 图像预处理（缩放/归一化） → PyTorch CNN推理 →输出检测结果（有人/无人） → 画面绘制标签与告警 → 日志记录 → 实时显示
```

3.2 自定义警戒区域

在监控画面中手动划定矩形禁区，仅对**框内区域**进行人体检测，框外目标忽略，降低误报率。

```
# 警戒区域参数（可自定义修改）
x1, y1 = 150, 100    # 矩形左上角坐标
w, h = 200, 250      # 矩形宽度、高度
cv2.rectangle(frame, (x1, y1), (x1+w, y1+h), (0,0,255), 2)
```

3.3 实时检测与告警逻辑

裁剪警戒区域图像，送入 CNN 模型推理：

- 检测到人体 → 画面显示红色告警文字 `WARNING: INTRUSION DETECTED`，记录告警日志
- 未检测到人体 → 画面显示绿色文字 `SAFE AREA`

3.4 运行方式

- 确保摄像头正常连接
- 执行主程序：

```
python main.py
```

- 画面自动弹出，显示监控画面 + 红色警戒框
- 人进入红框 → 实时告警；离开红框 → 恢复正常
- 按键盘 `q` 退出监控

3.5 运行成功输出

终端输出示例：

```
摄像头已开启，按 q 退出
[2026-05-13 15:30:20] SAFE AREA
[2026-05-13 15:30:25] WARNING: INTRUSION DETECTED
告警已记录到 logs/realtime.log
```

画面效果：红色矩形警戒框、实时状态文字、人体入侵时标红提示。

四、核心原理讲解

4.1 轻量 CNN 模型结构

采用适合物联网边缘设备的轻量卷积神经网络，参数量小、推理速度快：

4.2 警戒区域检测原理

- 划定矩形 ROI：过滤无关区域，聚焦重点监控范围
- ROI 裁剪：仅提取警戒区域图像，减少计算量
- 模型推理：CNN 学习目标等特征，区分“有/无”

- 告警触发：仅区域内检测到有目标存在时告警

4.3 图像预处理关键步骤

- 尺寸缩放：将 ROI 统一缩放到 256×256，适配模型输入
- 归一化：像素值映射到 [-1,1]，提升模型稳定性
- 通道转换：OpenCV 默认 BGR 格式，转为 RGB 适配 PyTorch

五、调试记录与常见问题

5.1 问题 1：检测误报率高

- 现象：无目标时频繁告警
- 原因：背景复杂、光线不足、检测阈值过低
- 解决：优化数据集（增加复杂背景样本）或者增加数据增强模式

5.2 问题 2：检测延迟高

- 现象：画面卡顿，告警响应慢
- 原因：模型过大、电脑配置低
- 解决：使用轻量 CNN、降低画面分辨率、减少推理频率

5.3 问题 3：警戒区域位置不合适

- 现象：框住无关区域或漏检重点区域
- 解决：修改代码中 `x1, y1, w, h` 参数，调整警戒框位置和大小

六、拓展实验

小实验 1：警戒区域自定义实验

- 任务：实现多警戒区域检测
- 操作：代码中新增多个矩形框，检测每个区域是否有人

小实验 2：使用摄像头实现实时监控区域

- 任务：对目标区域长时间实时监控
- 操作：打开摄像头(固定不动)，自主标记检测区域进行检测。

小实验3：不规则区域检测

- 任务：实现对不规则区域（以三角形为例）的检测
- 操作：设定一个三角形区域，检测这个区域内是否有人

七、实验报告要求

实验报告建议包含以下内容：

- 运行命令：启动监控程序的完整命令
- 运行结果：终端输出截图、监控画面截图（正常 + 告警状态）
- 流程简述：图像采集、区域划定、模型推理、告警输出的完整过程
- 完成拓展实验，附实验数据、结果截图与分析